

R: limpieza, manipulación y exploración de datos

Cheatsheet de joins en R

Jonatán Perren

13 de mayo de 2026

Un join combina dos tablas usando una **clave**: una columna (o conjunto de columnas) cuyos valores permiten emparejar filas. Forman parte de **dp`lyr`** (incluido en el tidyverse).

Mutating joins

— agregan columnas

- **inner_join()** — solo filas con coincidencia en ambas
- **left_join()** — todas las de la tabla izquierda
- **right_join()** — todas las de la tabla derecha
- **full_join()** — todas las de ambas tablas

Filtering joins

— filtran filas

- **semi_join()** — filas con coincidencia
- **anti_join()** — filas sin coincidencia

Sin clave

- **bind_rows()** — apila filas
- **bind_cols()** — apila columnas

¿Por qué necesitamos joins?

Los datos reales se distribuyen en múltiples tablas para evitar repetición. En `nycflights13`, el nombre de cada aerolínea se guarda una sola vez en `airlines`; `flights` solo guarda el código:

```
1 flights > count(carrier) # devuelve "UA", "AA", "DL"... códigos sin sentido
2
3 flights >
4   left_join(airlines, by = "carrier") >
5   group_by(name) >
6   summarise(demora_media = mean(arr_delay, na.rm = TRUE))
```

	name	demora_media
	<chr>	<dbl>
1	American Airlines Inc.	0.364
2	United Air Lines Inc.	3.56

Sin joins, el análisis queda con códigos crípticos. Los joins combinan tablas complementarias en el momento del análisis, sin necesidad de duplicar datos.

Datasets usados en este machete

Dataset	Descripción
<code>band_members</code>	integrantes de bandas: <code>name</code> , <code>band</code> . Incluido en <code>dplyr</code> .
<code>band_instruments</code>	instrumentos: <code>name</code> , <code>plays</code> . Incluido en <code>dplyr</code> .
<code>band_instruments2</code>	igual que <code>band_instruments</code> pero la columna clave se llama <code>artist</code> (no <code>name</code>).
<code>nycflights13::flights</code>	vuelos desde NYC en 2013: <code>tailnum</code> , <code>origin</code> , <code>year</code> , <code>month</code> , <code>day</code> , <code>hour</code> , ...
<code>nycflights13::planes</code>	datos de aeronaves: <code>tailnum</code> , <code>year</code> (fabricación), <code>manufacturer</code> , ...
<code>nycflights13::weather</code>	clima por hora y aeropuerto: <code>origin</code> , <code>year</code> , <code>month</code> , <code>day</code> , <code>hour</code> , ...

Los datasets `band_*` están diseñados para practicar joins: son pequeños y tienen casos sin coincidencia intencionales. `nycflights13` requiere `install.packages("nycflights13")`.

band_members

name	band
Mick	Stones
John	Beatles
Paul	Beatles

band_instruments

name	plays
John	guitar
Paul	bass
Keith	guitar

band_instruments2

artist	plays
John	guitar
Paul	bass
Keith	guitar

La clave es **name** (o **artist** en **band_instruments2**). **Mick** no tiene instrumento; **Keith** no tiene banda: son los casos sin coincidencia.

¿Qué filas incluye cada join?

Con `band_members` (x) y `band_instruments` (y):

Fila	inner	left	right	full	semi	anti
Mick (solo en x)	—	✓	—	✓	—	✓
John (en ambas)	✓	✓	✓	✓	✓	—
Paul (en ambas)	✓	✓	✓	✓	✓	—
Keith (solo en y)	—	—	✓	✓	—	—

`semi_join()` y `anti_join()` son filtering joins: solo conservan filas de x. Keith no puede aparecer porque no existe en `band_members`.

inner_join() — solo filas con coincidencia en ambas

```
1 band_members ► inner_join(band_instruments, by = "name")
```

	name	band	plays
	<chr>	<chr>	<chr>
1	John	Beatles	guitar
2	Paul	Beatles	bass

Mick (sin instrumento) y Keith (sin banda) quedan excluidos.

Usarlo cuando solo tienen sentido las filas con información completa en ambas tablas — por ejemplo, calcular el ratio entre dos indicadores que deben existir simultáneamente. Si falta uno, la fila no sirve.

left_join() – todas las filas de la tabla izquierda

```
1 band_members ► left_join(band_instruments, by = "name")
```

```
  name  band  plays
<chr> <chr> <chr>
1 Mick  Stones NA      # sin coincidencia: plays = NA
2 John  Beatles guitar
3 Paul  Beatles bass
```

Mick se conserva con **NA** en la columna que vino de la derecha. Keith queda excluido.

Es el join más frecuente en la práctica: enriquecer una tabla principal con datos de una tabla secundaria, conservando todas las filas originales.

right_join() — todas las filas de la tabla derecha

```
1 band_members > right_join(band_instruments, by = "name")
```

```
name band    plays
<chr> <chr>   <chr>
1 John Beatles guitar
2 Paul Beatles bass
3 Keith NA    guitar # sin coincidencia: band = NA
```

Keith se conserva con **NA** en band. Mick queda excluido.

Útil cuando la tabla principal está a la derecha — por ejemplo, al enriquecer una tabla de referencia completa con datos parciales de x. En la práctica, `right_join(x, y)` es equivalente a `left_join(y, x)`: se prefiere `left_join` invirtiendo el orden para mantener la tabla principal siempre a la izquierda.

full_join()

```
1 band_members > full_join(band_instruments, by = "name")
```

	name	band	plays
	<chr>	<chr>	<chr>
1	Mick	Stones	NA
2	John	Beatles	guitar
3	Paul	Beatles	bass
4	Keith	NA	guitar

`full_join()` es útil para **auditar** dos fuentes: muestra qué filas tienen coincidencia y cuáles quedan sin emparejar en cada tabla — los **NA** señalan exactamente dónde falta información.

semi_join() – filtrar filas con coincidencia

Filtering join: no agrega columnas. Solo conserva las filas de la tabla izquierda que tienen al menos una coincidencia en la derecha.

```
1 band_members ► semi_join(band_instruments, by = "name")
```

```
name band
<chr> <chr>
1 John Beatles
2 Paul Beatles
```

Usarlo cuando la condición de filtro está en otra tabla pero **no** se quieren agregar sus columnas al resultado – por ejemplo, conservar solo los vuelos de aerolíneas que operan más de 10 rutas, sin incorporar datos de `airlines` al tibble.

A diferencia de `inner_join()`, `semi_join()` nunca duplica filas aunque haya múltiples coincidencias en la tabla derecha.

anti_join() – filtrar filas sin coincidencia

Filtering join: conserva solo las filas de la tabla izquierda que **no** tienen coincidencia en la derecha.

```
1 band_members ► anti_join(band_instruments, by = "name")
```

```
  name  band  
<chr> <chr>  
1 Mick  Stones
```

Útil para detectar registros faltantes: integrantes sin instrumento, facturas sin pago, productos sin ventas en el período.

El argumento `by` =

Siempre especificar `by` explícitamente. Hay tres sintaxis:

```
1 band_members > left_join(band_instruments, by = "name")
2 band_members > left_join(band_instruments2, by = c("name" = "artist"))
3
4 band_members > left_join(band_instruments2, by = join_by(name == artist))
5
6
7 flights > left_join(weather,
8   by = join_by(origin, year, month, day, hour))
```

Sintaxis	Cuándo usarla
<code>by = "col"</code>	la clave tiene el mismo nombre en ambas tablas
<code>by = c("x_col" = "y_col")</code>	los nombres de la clave difieren entre tablas
<code>by = join_by(x_col == y_col)</code>	equivalente al anterior; preferida en dplyr 1.1+
<code>by = join_by(col1, col2)</code>	clave compuesta (varias columnas)

Si se omite `by`, dplyr detecta columnas con el mismo nombre en ambas tablas y las usa como clave (emite un mensaje, no un error). Es una fuente frecuente de bugs silenciosos: basta con que dos tablas compartan una columna por coincidencia para obtener un join incorrecto.

Columnas duplicadas – suffix =

Cuando ambas tablas tienen una columna con el mismo nombre (que no es la clave), `dplyr` agrega un sufijo para desambiguar.

```
1 flights > left_join(planes, by = "tailnum",  
2   suffix = c("_vuelo", "_fab"))
```

	tailnum	year_vuelo	year_fab	manufacturer	model
	<chr>	<int>	<int>	<chr>	<chr>
1	N14228	2013	1999	BOEING	737-824

Sin `suffix`, las columnas duplicadas se llaman `year.x` y `year.y`. Siempre especificar sufijos descriptivos. En `flights` y `planes`, la columna `year` tiene significados distintos (año del vuelo vs. año de fabricación).

Claves duplicadas — relationship =

Si la clave no es única en la tabla derecha, el join multiplica filas (producto cartesiano).
relationship permite verificar la relación esperada.

```
1 flights > left_join(airlines, by = "carrier",  
2   relationship = "many-to-one")
```

Valor	Significado
"one-to-one"	la clave es única en ambas tablas
"one-to-many"	única en x, puede repetirse en y
"many-to-one"	puede repetirse en x, única en y (el más frecuente)
"many-to-many"	puede repetirse en ambas (dplyr advierte si no se declara)

Si el resultado tiene más filas de las esperadas, hay claves duplicadas en la tabla derecha.
Usar **anti_join()** o **count()** para diagnosticar.

bind_rows() y bind_cols()

Combinan tablas **sin una clave**: por posición, no por valores.

```
1 bind_rows(enero, febrero, marzo) # apila filas (deben tener las mismas columnas)
2 bind_cols(datos_x, datos_y)      # pega columnas (deben tener el mismo n de filas)
```

Función	Dirección	Cuándo usarla
<code>bind_rows()</code>	vertical	mismas columnas, más observaciones
<code>bind_cols()</code>	horizontal	mismas filas, más variables

`bind_cols()` no verifica que las filas correspondan: es responsabilidad del programador que el orden sea correcto. Para combinar por valores usar un join.

Función	Tipo	Filas en el resultado
<code>inner_join()</code>	mutating	solo las que coinciden en ambas tablas
<code>left_join()</code>	mutating	todas las de x; NA donde no hay coincidencia en y
<code>right_join()</code>	mutating	todas las de y; NA donde no hay coincidencia en x
<code>full_join()</code>	mutating	todas las de ambas; NA donde no hay coincidencia
<code>semi_join()</code>	filtering	las de x que tienen coincidencia en y (sin columnas nuevas)
<code>anti_join()</code>	filtering	las de x que no tienen coincidencia en y (sin columnas nuevas)

Argumento	Uso
<code>by</code>	clave de unión (siempre explícito)
<code>suffix</code>	sufijos para columnas con nombre duplicado (default: <code>.x / .y</code>)
<code>relationship</code>	verifica la cardinalidad esperada del join